实验三：校园二手集市桌面程序实现报告

目录

1. 项目说明
2. 已有 UML 到代码的实现
3. 程序结构和主要实现
4. 源代码规模与统计口径
5. 大模型辅助和人工审查
6. 编译、运行和测试
7. Git 与 GitHub 远程管理
8. 阶段性结论
9. 数据库、事务与权限处理
10. 界面调用过程与演示方式
11. 当前范围与后续改进

项目说明

我完成的这个实验主要实现的是一个校园二手交易的小型桌面程序。程序运行在本机，不依赖网络服务。当前版本使用 Python、SQLite、pywebview 和 HTML/CSS/JavaScript：Python 负责数据、权限和交易规则，WebView 页面负责界面展示和用户操作。

程序中我提前预置了买家、卖家和管理员三种身份，这样我觉得更加便于测试和使用，买家可以浏览、收藏和预约商品；卖家可以发布、编辑和下架商品，并处理收到的预约；管理员可以审核商品、处理举报、查看统计和操作日志。项目根目录中的 `docs/运行与验收说明.md` 列出了这些验收点，实际业务逻辑集中在 `code/market_service.py`。

如果想要启动查看的话，请你注意当前启动方式以 `code/README.md` 和最近提交的代码为准：

```
cd code
python web_desktop.py
```

也可以双击 `code/启动校园二手集市.bat`。首次启动时，程序会在 `code` 目录创建本地 SQLite 演示数据库 `market.db`。该数据库是运行生成物，不属于源代码提交内容。

已有 UML 到代码的实现

我依据老师布置的作业要求，这个项目的 UML 图已在实验二完成，实验三的重点是按照已有设计完成程序实现，而不是重新绘制 UML 图。本次实验目录中的 `UML` 文件夹没有另外放入图源文件；远程仓库已有的实验二目录保存了相关 UML 交付材料。

角色和实体的对应

我完成的这个程序把用户、商品、预约、收藏、举报和操作日志作为核心数据。`market_service.py` 中的 `User` 数据类表示登录后的用户；`Roles` 定义买家、卖家和管理员三种角色。SQLite 初始化代码建立了 `users`、`products`、`reservations`、`favorites`、`reports` 和 `operation_logs` 六张表。

当前这种划分对应已有 UML 中的领域对象和关系：用户发布商品，商品可以被买家预约；预约记录买卖双方的交易过程；收藏和举报是商品的附属业务；日志用于记录关键操作。表之间通过用户和商品的编号关联，数据库连接开启了外键约束。

设计对象或关系	代码位置	实现方式
用户与角色	`User`、`Roles`、`users` 表	登录后保存用户编号、手机号、昵称和角色；写操作先检查角色。
商品	`products` 表、`create_product`、`update_product`	保存标题、描述、分类、价格、状态、版本号等字段。
买家预约商品	`reservations` 表、`reserve`	预约时同时检查商品状态和版本号，再建立预约记录。
卖家处理预约	`confirm_reservation`、`confirm_delivery`、`cancel_or_reject`	把待确认、已确认、交付、取消或拒绝等操作限制在正确角色和正确状态下。
收藏与举报	`favorites`、`reports` 表及对应方法	防止收藏自己的商品，举报需要填写原因，管理员负责处理。
管理操作和统计	`operation_logs`、`statistics`、`logs`	记录重要业务动作，并按商品状态统计数量。

状态转换的实现

为了让我这个项目的状态清晰一些，我特意设计了有限状态机，商品状态由 `ProductStatus` 统一定义，包括“草稿”“待审核”“在售”“已预约”“已售出”“已下架”和“违规下架”。预约状态由 `ReservationStatus` 定义，包括“待确认”“已确认”“已取消”“已拒绝”和“已完成”。

实际代码没有只在界面层改变显示文字，而是在服务层限制状态变化。例如：

1. 卖家保存商品时状态为“草稿”，提交后变为“待审核”。
2. 管理员审核通过后，商品才会变为“在售”；驳回后回到“草稿”。
3. 买家预约“在售”商品后，商品变为“已预约”，同时生成“待确认”的预约。
4. 卖家确认预约、确认交付后，买家才可以确认收货；收货后预约变为“已完成”，商品变为“已售出”。
5. 买家取消预约或卖家拒绝预约时，商品恢复为“在售”。

预约方法还接收商品的 `expected_version`。如果页面上的版本已经过期，或者商品已经被其他人预约，更新会被拒绝。这一部分把已有状态机中的交易约束落实为可执行的检查，也避免旧页面再次提交时覆盖最新状态。

从业务服务到桌面界面

为了让整体的业务逻辑更加清晰，我专门让AI帮忙设计了`web_desktop.py` 中的 `MarketApi` 是界面和业务服务之间的一层接口。页面不直接读写数据库，而是通过 `window.pywebview.api` 调用 Python 方法。接口把成功结果包装为 `{ok: true, data: ...}`，业务错误包装为 `{ok: false, message: ...}`，这样页面可以统一显示提示。

`code/ui/index.html` 包含登录、首页、商品详情、收藏、预约、卖家商品、发布商品、个人中心和管理工作台等页面逻辑。前端负责收集输入、切换页面和渲染列表；注册、审核、预约、举报、确认收货等写操作仍会回到 `MarketService` 进行校验。因此即使前端传入不合适的数据，核心规则仍由 Python 服务控制。

程序结构和主要实现

当前实际提交并推送的 WebView 版本由下面四个源文件组成：

```
code/
├── market_service.py       # 领域服务、SQLite 表、权限、状态机和日志
├── test_core_flow.py       # 不依赖界面的核心流程自检
├── web_desktop.py          # pywebview 桌面窗口与 Python API
└── ui/
    └── index.html          # HTML、CSS、JavaScript 页面
```

其中，`market_service.py` 是程序的核心。它使用事务上下文管理器来确保一次业务操作要么完整写入，要么在出错时回滚。密码不会以明文写入数据库，而是使用 SHA-256 摘要保存。这个实现更适合本地演示程序；它不是面向真实互联网业务的完整账号安全方案。

测试文件 `test_core_flow.py` 使用单独的 `test_market.db`。测试开始前会删除旧测试库，结束前会删除测试库，因此不会改变正常运行时的 `market.db`。测试覆盖了商品预约和旧版本拦截、卖家确认预约和交付、买家确认收货、管理员审核、商品编辑、收藏、举报处理、下架权限以及昵称修改等流程。

源代码规模与统计口径

代码行数按照当前已经推送到 GitHub 的 WebView 版本统计。统计只包含业务服务、核心测试、桌面桥接和当前页面这四个源文件；不包含 Markdown 说明、批处理启动脚本、SQLite 数据库、`__pycache__`、测试数据库、截图、PDF、ZIP 和其他未作为当前入口使用的旧界面原型。

我计算的时候依据实验的要求，没有把 GUI 自动生成代码算入。当前统计的 `ui/index.html` 和 `web_desktop.py` 是本项目当前手写的 WebView 页面和桥接代码；目录中的旧 Tkinter 原型、设计稿裁切脚本及图片资产没有被纳入统计，也没有进入本次三次提交。

可在项目根目录用下面的 PowerShell 命令复现统计：

```powershell
$files = @(
  ".\code\market_service.py",
  ".\code\test_core_flow.py",
  ".\code\web_desktop.py",
  ".\code\ui\index.html"
)

foreach ($file in $files) {
  $lines = (Get-Content -LiteralPath $file | Measure-Object -Line).Lines
  "{0}`t{1}" -f $file, $lines
}

($files | ForEach-Object {
  (Get-Content -LiteralPath $_ | Measure-Object -Line).Lines
} | Measure-Object -Sum).Sum
```

本次实际统计结果如下：

文件	行数
`code/market_service.py`	606
`code/test_core_flow.py`	92
`code/web_desktop.py`	183
`code/ui/index.html`	69
合计	**950**

`index.html` 的行数相对少，是因为样式和部分前端函数采用了压缩式排版；这不表示页面功能很少。行数只用于说明当前代码规模，不能单独代表功能复杂度或代码质量。

大模型辅助和人工审查

在我和大模型合作完成这次实验的过程中，充分感受到这个AI的能力是真的强悍，但是我发现以下:

大模型在本次工作中主要用于阅读现有代码、归纳模块职责、检查提交范围和整理验证命令。例如，在准备远程提交时，先读取了 `market_service.py`、`test_core_flow.py`、`web_desktop.py` 和当前页面，才将代码拆为三个层次；在编写本文档时，再根据源文件、测试脚本和 Git 记录逐项核对。

我在实验过程中认为人工需要保留的判断主要有以下几类：

1. **确定当前入口。** 工作区的旧说明中仍出现了 `secondhand_market.py`，但当前 `code/README.md` 和最新提交使用的是 `web_desktop.py`。报告采用后者作为当前版本入口，同时保留这一差异的说明，而没有擅自修改原说明文件。
2. **确定提交范围。** `market.db`、`test_market.db`、`__pycache__`、截图、PDF 和 ZIP 都是生成物或交付材料，不能和源码一起提交。实际提交范围只包括当前应用的源码、启动脚本和运行说明。
3. **检查业务规则。** 大模型可以帮助逐段阅读状态判断，但是否允许某个角色操作、状态能否转换，仍需要按照验收说明和测试场景进行核对。例如，预约时的版本校验、卖家交付后才能由买家收货等规则，均由测试脚本验证。
4. **检查远程历史。** 远程仓库已有实验二内容，不能把本地目录直接覆盖到远程。实际操作先确认远程分支和历史，再在 `main` 的现有提交后追加三个新提交，整个过程没有使用强制推送。

因此，大模型在这里更适合承担代码阅读、命令整理和重复检查的工作；需求取舍、提交目标和最终结果仍应通过实际文件、命令输出和人工判断确认。

编译、运行和测试

编译检查

本次对三个 Python 源文件执行了编译检查：

```
python -m py_compile `
  .\code\market_service.py `
  .\code\test_core_flow.py `
  .\code\web_desktop.py
```

结果为通过。该检查可以发现 Python 语法错误，但不会启动桌面窗口。

核心流程自检

在 `code` 目录运行：

```
python .\test_core_flow.py
```

测试进程退出码为 `0`，并输出“核心交易、商品管理、收藏、举报与权限流程自检通过。”测试中实际执行了以下关键过程：

- 买家预约数码商品，并验证旧版本的重复预约被拒绝；
- 卖家确认预约、确认交付，买家确认收货，商品最终变为“已售出”；
- 管理员审核待审核商品，检查商品上架和日志数量；
- 卖家保存并编辑草稿；
- 买家收藏和取消收藏，提交举报，管理员驳回举报；
- 卖家下架商品，并验证普通用户不能查看下架商品；
- 买家修改昵称。

页面脚本检查

页面中的 JavaScript 不是单独文件，因此先从 `code/ui/index.html` 提取 `<script>` 内容，再执行：

```
node --check market-ui-syntax-check.js
```

检查退出码为 `0`。临时检查文件在检查后已删除。该项检查说明脚本语法可被 Node.js 解析；页面交互仍需要在本机桌面窗口中结合演示账号进行人工操作验证。

Git 与 GitHub 远程管理

本项目使用的远程仓库地址为：(还是之前的那个仓库)

```
https://github.com/HarryPotter-jun/second-hand-trading-system-lab1-rebuild.git
```

工作分支为 `main`。远程仓库已有实验二历史，所以本次没有新建远程仓库，也没有重写历史。三个提交都建立在原有 `main` 的基础上，并且每完成一次提交就分别执行一次普通 `git push origin main`。

顺序	Commit	提交信息	分阶段理由
1	`35a89dd4fb5d7c96c4a667cbb035af409fceb42c`	`feat: add marketplace domain service and core flow test`	先提交不依赖桌面界面的领域服务和核心自检，保证后续界面有可验证的业务基础。

顺序	Commit	提交信息	分阶段理由
2	`786e7b3e43d9d01f9fd2252949d224d78ab17e92`	`feat: add WebView desktop bridge and launcher`	再加入 Python 到 WebView 的桥接、启动脚本和当前入口说明，使程序可以作为本地桌面应用启动。
3	`d04740173efd71ec5ae6883d1f7706e9461504e4`	`feat: add interactive campus marketplace WebView UI`	最后加入页面样式和交互逻辑，使界面通过桥接接口访问已经存在的业务服务。

三个提交都包含 `Co-authored-by: Copilot App <223556219+Copilot@users.noreply.github.com>` trailer。推送完成后使用 `git ls-remote origin refs/heads/main` 对照本地 `HEAD`，二者均为：

```
d04740173efd71ec5ae6883d1f7706e9461504e4
```

同时对这三个提交相对于原远程基线的文件列表进行了检查。提交范围仅包含 `code/README.md`、三个 Python 源文件、`code/ui/index.html` 和启动批处理文件，没有包含 SQLite 数据库、缓存、临时测试库、PDF、ZIP、实验报告、UML 或其他文档。

阶段性结论

本项目把校园二手交易中较常见的用户角色、商品审核、预约交易、收藏、举报和日志功能放在一个本地桌面程序中实现。实现时把业务规则放在 `MarketService`，把桌面窗口和页面操作放在桥接层及前端页面中，这样测试核心业务时不需要依赖图形界面。

从当前测试结果看，预约、状态变化、权限限制、审核、举报和日志等主要流程能够正常执行。后续如果继续完善，可以补充更细的输入校验、页面人工验收记录，以及统一旧说明文件和当前 WebView 入口的表述。

数据库、事务与权限处理

表的职责

为了数据的持续性，我这个程序使用一个本地 SQLite 数据库保存演示数据。数据库初始化代码位于 `MarketService.initialize()`，使用 `CREATE TABLE IF NOT EXISTS` 创建表，因此第一次运行可以建立表结构，后续运行不会重复创建。首次初始化后，`seed_demo_data()` 会在用户表为空时写入演示账号、商品和一条待处理举报。

各表的用途如下：

表名	主要内容	在程序中的作用
`users`	手机号、昵称、密码摘要、角色、创建时间	登录、注册和权限判断。

表名	主要内容	在程序中的作用
`products`	商品描述、价格、商品状态、版本号、审核或违规原因	商品列表、详情、发布、审核和下架。
`reservations`	商品编号、买家编号、预约状态、交付和收货时间	保存一次预约交易的进度。
`favorites`	买家编号、商品编号、收藏时间	保存买家收藏的商品，并限制同一买家不能重复收藏同一商品。
`reports`	举报人、商品、举报原因、处理状态和处理结果	由管理员处理商品举报。
`operation_logs`	操作人、动作、对象、结果、原因和时间	记录登录、预约、审核、举报等关键操作。

`products` 表中的 `version` 字段是交易流程中的一个关键字段。商品被编辑、预约、下架、审核或确认售出时，代码会增加版本号。预约请求带着用户打开页面时看到的版本号进入服务层；如果这个版本号已经不是数据库中的最新值，`reserve()` 就会返回“商品状态已变化，请刷新后再试”。这样做不能代替真正高并发系统中的完整锁机制，但对于本地演示程序，能够清楚地表现出防止旧状态覆盖新状态的处理思路。

事务处理

每次业务查询或修改都通过 `transaction()` 打开 SQLite 连接。该方法设置 `PRAGMA foreign_keys = ON`，并在正常执行后提交；出现异常时回滚；最后关闭连接。这样，预约商品时“修改商品状态”和“插入预约记录”会放在同一事务中，不会出现商品已经显示为已预约但没有预约记录的中间状态。

类似地，取消或拒绝预约时，代码先更新预约状态，再把商品从“已预约”恢复为“在售”；确认收货时，代码更新预约的完成时间，并把商品改为“已售出”。这些操作都在同一个事务内完成。核心测试中完整走过了这几条路径，能够验证状态变化在当前实现下是连贯的。

权限检查

根据我对一些软件的理解和一些小经验，权限判断不只放在页面按钮上，也在服务方法内执行。`_require_role()` 是较常用的角色检查方法，例如创建、编辑和下架商品必须是卖家，审核和查看统计必须是管理员，收藏和预约必须是买家。

还有一些检查与具体数据有关：

- 买家不能预约或收藏自己发布的商品；
- 已下架或违规下架的商品不能被普通用户查看或收藏；
- 已预约商品的详情只允许交易双方或管理员查看；
- 只有商品的发布者可以编辑或下架该商品；
- 只有对应买家可以取消预约和确认收货；
- 只有对应卖家可以确认、拒绝预约或确认交付；
- 驳回商品审核、处理举报时必须给出原因。

前端页面会根据当前角色隐藏不适用的操作入口，但真正的权限结论以服务层为准。这种做法使页面显示和业务规则相互配合，即使以后更换界面，实现也不需要把同一套规则再写一遍。

界面调用过程与演示方式

本地调用过程

程序启动后，`web_desktop.py` 创建一个 pywebview 窗口，并把 `ui/index.html` 作为本地文件加载。页面中的 `api()` 返回 `window.pywebview.api`，每次点击需要读写数据的功能时，JavaScript 使用异步方式调用 Python 侧的 `MarketApi` 方法。

例如，买家在商品详情页点击“立即预约”后，页面把商品编号和页面保存的版本号传给 `reserve(productId, version)`；`MarketApi.reserve()` 再调用 `MarketService.reserve()`。如果服务层成功完成预约，页面跳转到“我的预约”；如果服务层抛出业务错误，桥接层把错误信息返回，页面用统一的提示框显示。收藏、商品发布、商品审核、举报处理和确认收货也使用同样的模式。

这样划分后，前端没有直接拼接 SQL，也没有直接决定商品状态。页面主要负责布局、列表渲染、表单输入和提示；数据库操作都保留在 Python 服务中。对于当前实验规模，这个结构比把所有逻辑写在页面脚本中更容易检查。

演示账号

程序预置了以下本地演示账号，账号信息同时写在 `code/README.md` 和界面登录页中：

角色	手机号	密码	可重点演示的功能
买家	`13800000001`	`buyer123`	浏览、收藏、预约、取消预约、确认收货、举报。
卖家	`13800000002`	`seller123`	发布或编辑商品、查看收到的预约、确认预约和交付、下架。
管理员	`13800000003`	`admin123`	审核待审核商品、处理举报、查看统计和最近操作日志。

一次较完整的人工演示可以先用买家账号浏览“数码”分类中的在售商品并预约；退出后以卖家账号确认预约和交付；再切换回买家账号确认收货。需要展示审核功能时，可以使用卖家账号提交商品，然后以管理员账号到管理工作台审核。测试脚本已经验证了同一类状态变化，但界面演示可以补充说明每一步在页面中的呈现。

当前范围与后续改进

当前程序是课程实验的本地演示版本，目标是把主要业务规则跑通，目前还有很多等我以后需要优化的地方，而不是提供生产环境的交易平台。下面几项属于当前实现的范围限制：

1. 数据库是单机 SQLite 文件，适合演示和测试，不包含服务器部署、备份、多人并发访问或账号找回功能。
2. 商品图片使用图标表示，程序没有实现真实图片上传、文件校验或图片存储。
3. 登录密码使用 SHA-256 摘要保存，避免了明文保存，但没有加入随机盐、密码重置和登录频率限制等生产系统常见能力。
4. 交易仅记录预约、交付和收货状态，不包含在线支付、物流查询或第三方身份认证。
5. 当前核心自动化测试覆盖服务层主要流程；页面脚本已做语法检查，但完整的鼠标点击和页面布局验收仍需要在本机窗口中人工完成。

如果后续继续完善，我会优先补充两个方面：一是把旧的运行说明统一为当前 WebView 入口，避免阅读文档时混淆；二是围绕异常输入和权限边界增加更多独立测试，例如重复举报、卖家处理已完成预约、管理员重复处理同一举报等。这样可以让程序的可验证范围更完整，而不会只依赖界面演示。

结语

我在完成实验三的过程中，充分的对照看了看实验二已经完成的 UML，尽量努力把已有的用户、商品、预约和管理关系做成了可以运行的本地程序。我为了保证项目的清晰性，特意设计当前实现为把数据规则放在 Python 服务层，把桌面窗口和交互页面分开，并用 SQLite 保存演示数据。通过编译检查、核心流程自检和页面脚本语法检查，可以确认当前提交的代码具备基本的可运行性。

我在写这个实验报告中的代码规模、命令、测试结果、远程地址和提交编号都来自当前工作区或实际命令输出。对于没有实际验证过的页面操作和未在本实验中重新制作的 UML 图，报告没有补造截图或虚构过程。